


```
!nvidia-smi
import torch, math
print("Torch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
```

Sat Nov 29 01:08:49 2025

```
+-----+
| NVIDIA-SMI 550.54.15                  Driver Version: 550.54.15          CUDA Versi
|-----+-----+-----+
| GPU   Name           Persistence-M   Bus-Id        Disp.A   | Volatile
| Fan   Temp    Perf       Pwr:Usage/Cap |      Memory-Usage   | GPU-Util
|-----+-----+-----+-----+-----+
|    0   Tesla T4               Off          00000000:00:04.0 Off   |
| N/A    34C     P8              10W / 70W   |      0MiB / 15360MiB |      0%
|-----+-----+-----+-----+-----+
+-----+
```

```
+-----+
| Processes:
| GPU   GI    CI          PID    Type    Process name
|-----+-----+-----+-----+
| No running processes found
+-----+
```

```
Torch version: 2.9.0+cu126
CUDA available: True
```

```
import torch, math
import matplotlib.pyplot as plt

# Use GPU if available (fine to stay on CPU for L=1)
device = 'cuda' if torch.cuda.is_available() else 'cpu'
dtype = torch.float64

# SU(2) constants
N = 2
C_V = 6.0 / N # from the Wilson Hessian bound
c0 = (N**2 - 1.0) / (2.0 * N) # Haar mass coefficient from the cleaned pr

print("device:", device)
print("c0 =", c0, "C_V =", C_V)
```

```
device: cuda
c0 = 0.75 C_V = 3.0
```

```
# Build Pauli matrices on the chosen device
def build_paulis(device):
```

```

sigma1 = torch.tensor([[0, 1],
                        [1, 0]], dtype=torch.complex128, device=device)
sigma2 = torch.tensor([[0, -1j],
                        [1j, 0]], dtype=torch.complex128, device=device)
sigma3 = torch.tensor([[1, 0],
                        [0, -1]], dtype=torch.complex128, device=device)
return torch.stack([sigma1, sigma2, sigma3], dim=0)  # (3,2,2)

paulis = build_paulis(device)

def su2_from_vec(a_vec, paulis):
    """
    a_vec: (... , 3) real tensor (Lie algebra coordinates)
    paulis: (3,2,2) Pauli matrices
    returns: (... , 2, 2) complex SU(2) matrices via  $\exp(i * a \cdot \sigma / 2)$ 
    """
    a = a_vec.to(dtype=torch.float64, device=paulis.device)
    a_c = a.to(paulis.dtype)  # complex
    # A = (a_j sigma_j / 2)
    A = (a_c.unsqueeze(-1).unsqueeze(-1) * (paulis / 2.0)).sum(dim=-3)
    M = 1j * A  # i*A

    orig_shape = M.shape[:-2]
    M_flat = M.reshape(-1, 2, 2)
    U_flat = torch.zeros_like(M_flat)
    for idx in range(M_flat.shape[0]):
        U_flat[idx] = torch.matrix_exp(M_flat[idx])
    U = U_flat.reshape(*orig_shape, 2, 2)
    return U

# quick sanity check:  $U^\dagger U \sim I$ 
test_vec = torch.randn(3, device=device)
U_test = su2_from_vec(test_vec, paulis)
print("U^\dagger U at test point:\n", (U_test.conj().transpose(-1,-2) @ U_test).rea

```

```

U^\dagger U at test point:
tensor([[1.0000e+00, 1.1053e-16],
        [1.1053e-16, 1.0000e+00]], device='cuda:0', dtype=torch.float64)

```

```

def make_links(A, a_lat, g, paulis):
    """
    A: (L,L,L,L,4,3) real Lie algebra field
    returns U: (L,L,L,L,4,2,2) complex link matrices
    """
    L = A.shape[0]
    A_flat = (a_lat * g * A).reshape(-1, 3)
    U_flat = su2_from_vec(A_flat, paulis)
    U = U_flat.reshape(L, L, L, L, 4, 2, 2)
    return U

```

```

def wilson_action(A, a_lat, g, beta, paulis):
    """
    Wilson action  $S_W = \sum_p (1 - (1/N) \text{Re Tr } U_p)$ ,  $N=2$ .
    """
    L = A.shape[0]
    U = make_links(A, a_lat, g, paulis)
    S = torch.zeros(), dtype=dtype, device=A.device)

    for mu in range(4):
        for nu in range(mu+1, 4):
            for x0 in range(L):
                for x1 in range(L):
                    for x2 in range(L):
                        for x3 in range(L):
                            xs = [x0, x1, x2, x3]

                            #  $U_\mu(x)$ 
                            U1 = U[x0, x1, x2, x3, mu]

                            #  $U_\nu(x+\mu)$ 
                            xp = xs.copy()
                            xp[mu] = (xp[mu] + 1) % L
                            U2 = U[xp[0], xp[1], xp[2], xp[3], nu]

                            #  $U_\mu^\dagger(x+\nu)$ 
                            xq = xs.copy()
                            xq[nu] = (xq[nu] + 1) % L
                            U3 = U[xq[0], xq[1], xq[2], xq[3], mu].conj().tr

                            #  $U_\nu^\dagger(x)$ 
                            U4 = U[x0, x1, x2, x3, nu].conj().transpose(-1, -

                            U_p = U1 @ U2 @ U3 @ U4
                            tr = torch.trace(U_p)
                            S = S + (1.0 - 0.5 * tr.real) #  $1 - (1/2) \text{ReTr}$ 

    return beta * S

def haar_action(A, a_lat, g, c0):
    """
    Quadratic Haar term:  $(c0/2) a^2 g^2 \sum_b ||A_b||^2$ 
    """
    return 0.5 * c0 * (a_lat**2) * (g**2) * (A**2).sum()

def effective_action(A, a_lat, g, beta, c0, paulis):
    return wilson_action(A, a_lat, g, beta, paulis) + haar_action(A, a_lat,

```

```

from torch.autograd.functional import hessian

```

```

def hessian_at_point(A, a_lat, g, beta, c0, device='cpu', x0=None):

```

```

def hessian_at_point(L, a_lat, g, beta, c0, device=cpu, x0=None):
    """
    Compute Hessian of S_eff wrt all Lie-algebra coordinates on an L^4 lattice
    x0: optional starting point (flattened), else uses zero field.
    Returns sorted eigenvalues as a 1D tensor.
    """
    paulis = build_paulis(device)
    n_params = L * L * L * L * 4 * 3

    def S_from_flat(flat):
        A = flat.view(L, L, L, L, 4, 3)
        return effective_action(A, a_lat, g, beta, c0, paulis)

    if x0 is None:
        x0 = torch.zeros(n_params, dtype=dtype, device=device, requires_grad=True)
    else:
        x0 = x0.clone().detach().to(dtype).to(device)
        x0.requires_grad_(True)

    H = hessian(S_from_flat, x0)
    evals, _ = torch.linalg.eigh(H.detach().cpu())
    return evals

def theoretical_rho_star(a_lat, g):
    beta = 2.0 * N / (g**2)
    return c0 * (a_lat**2) * (g**2) - beta * C_V

```

```
L = 1
a_lat = 1.0
g = 3.0
beta = 2.0 * N / (g**2)

evals = hessian_at_point(L, a_lat, g, beta, c0, device=device)
rho = theoretical_rho_star(a_lat, g)

print("Eigenvalues at A=0:", evals.numpy())
print("λ_min(H) =", evals[0].item())
print("ρ_*(a) =", rho)
```

```
Eigenvalues at A=0: [6.75 6.75 6.75 6.75 6.75 6.75 6.75 6.75 6.75 6.75 6.75 6.75]
λ_min(H) = 6.750000000000001
ρ*(a) = 5.416666666666667
```

```
def scan_g_values(g_values, L=1, a_lat=1.0, device=device):
    results = []
    for g in g_values:
        beta = 2.0 * N / (g**2)
        rho_star = theoretical_rho_star(a_lat, g)
```

```

    evals = hessian_at_point(L, a_lat, g, beta, c0, device=device)
    lam_min = evals[0].item()
    results.append((g, rho_star, lam_min))
    print(f"g={g:.3f} rho_*={rho_star:.4f} lambda_min={lam_min:.4f}")
return results

g_vals = [1.5, 2.0, 2.5, 3.0]
results = scan_g_values(g_vals, L=1, a_lat=1.0, device=device)

```

```

g=1.500 rho_*=-3.6458 lambda_min=1.6875
g=2.000 rho_*=0.0000 lambda_min=3.0000
g=2.500 rho_*=2.7675 lambda_min=4.6875
g=3.000 rho_*=5.4167 lambda_min=6.7500

```

```

def plot_scan(results):
    gs      = [r[0] for r in results]
    rhos    = [r[1] for r in results]
    lambdas = [r[2] for r in results]

    plt.figure()
    plt.plot(gs, rhos, 'o-', label=r'theoretical $\rho_*(a)$')
    plt.plot(gs, lambdas, 's-', label=r'numerical $\lambda_{\min}$')
    plt.xlabel('g')
    plt.ylabel('curvature / eigenvalue')
    plt.grid(True)
    plt.legend()
    plt.show()

plot_scan(results)

```



